

ADLYTIC_MASTER_ARCHITECT_AUDIT_2026

Document type: Production-grade master architecture audit Repository: /Users/aliahhed/Downloads/adlytic Audit date: 2026-06-29 Stack reality: Hono + @hono/node-server, Prisma 7 + PostgreSQL (@prisma/adaptor-pg + pg.Pool), server-rendered TypeScript HTML pages (layout.ts, *Page.ts), inline SHARED_JS, background workers in-process (serve.ts), Meta Graph API client, Stripe billing, Claude AI layer NOT Next.js — no React SSR, no App Router, no Vercel edge functions

Scope: Full codebase review across five pillars with surgical focus on prisma/schema.prisma, syncAccount.ts, serve.ts, server.ts, metaClient.ts, layout.ts, checkWorkspaceTokenHealth.ts, Meta/Stripe webhook handlers, and Postgres advisory-lock usage.

Commits reviewed through: 885f009 (UI resilience / sync UX / token-health), f7b85af (SyncJob dedupe + advisory lock cleanup), ec1f4a5 (Meta webhooks), 26858ed (real sync polling B-0).

Executive Summary

Adlytic is a mature Phase-1 ads intelligence platform with unusually strong domain discipline: Meta transport is isolated in metaClient.ts, dashboard DTO assembly is centralized in getDashboard.ts, ETL is idempotent via upserts, and recent commits (885f009, f7b85af, 26858ed) show active hardening of sync UX, token health, and webhook security.

However, the system carries three production-critical structural risks that a master architect must treat as first-class:

1. Session-scoped Postgres advisory locks + Prisma connection pooling — Locks are acquired with `pg_try_advisory_lock` (session-scoped, not transaction-scoped). Prisma's pool can reuse connections across requests/workers. `syncChunked` mitigates leaked locks via `pg_advisory_unlock_all()` in a pinned transaction (f7b85af), but `sync()` (auto-sync path) lacks this mitigation, and `unlock_all` itself is a blunt instrument that can release unrelated locks if the same connection ever holds multiple advisory keys.

1. Single-process background orchestration at scale — Auto-sync, token refresh, raw-insight pruning, rollout refresh, fire-and-forget `syncChunked`, Meta webhook debounced reconciles, and engine/brain runs all share one Node event loop and one Prisma pool. There is no job queue, no worker isolation, and no backpressure beyond serial account iteration and Meta politeness delays. A large account backfill or Meta code-17 storm can starve HTTP latency and accumulate PENDING/PROCESSING SyncJobs.

1. Multi-instance deployment gaps — OAuth CSRF state is correctly persisted (OAuthState model), but post-callback OAuth sessions remain in-memory (oauthSessions Map in server.ts). Rate-limit maps, platform-stats cache, Meta webhook debounce timers, and the auto-sync coarse lock (adlytic:auto-sync) are per-process. Railway multi-replica deploys will exhibit split-brain OAuth completion, duplicated background sync attempts (partially mitigated by advisory lock), and inconsistent rate-limit enforcement.

Positive highlights:

- Meta webhook HMAC verification is correctly implemented (raw body, length-guarded timingSafeEqual, async decoupling) — mirrors Stripe pattern.
- Token encryption uses AES-256-GCM with distinct TokenDecryptError — never conflated with Meta 190.
- SyncJob polling UX (layout.ts pollSyncJob, chunksDone/chunksTotal progress, reused active-job dedupe) is production-minded.
- Stripe webhook idempotency via ProcessedStripeEvent PK = event.id inside \$transaction.
- Config centralization (config.ts) with production fail-fast for secrets.

Overall verdict: Shippable for early production / controlled beta with a single Railway instance and moderate account count. Not yet architecturally ready for multi-replica horizontal scale, high account cardinality, or strict SLAs without addressing locking semantics, job queue extraction, and index/ops gaps documented below.

Audit Methodology

Step | Action

- 1 | Static analysis of ~45 src/ TypeScript modules, Prisma schema (30+ models), migrations
- 2 | Trace critical paths: OAuth connect !' kickoffInitialSync !' syncChunked !' engines !' dashboard poll

- 3 | Trace failure paths: Meta 190, code 17, token decrypt, advisory lock skip, SyncJob FAILED
- 4 | Review webhook security contracts (Stripe + Meta)
- 5 | Inspect UI inline JS (SHARED_JS, dashboard/campaigns onboarding poll from commit 885f009)
- 6 | Cross-reference internal docs (HOW_IT_WORKS.md, META_INTEGRATION_SPEC.md) vs code
- 7 | Count : any / as any in src/ (31 occurrences across 12 files)

Severity scale:

Level | Meaning

- Critical | Data loss, security bypass, permanent lockout, or production outage likely under normal load
- High | Significant correctness, integrity, or availability risk; workaround exists but fragile
- Medium | Degraded UX, ops burden, or latent failure under scale
- Low | Style, minor inefficiency, or future tech debt

Pillar 1 — Architectural Patterns & Code Quality

1.1 System topology (as-built)

```
Browser % % % Hono (server.ts) % % % Prisma/pg.Pool % % % PostgreSQL
% % % %2
% % % %
% % % SSR pages (*Page.ts + layout.ts SHARED_JS)
% % % REST /api/*
% % % setImmediate background:
% % %   syncChunked, runEngines, runBrainOrchestrator,
% % %   processMetaWebhookEvent, kickoffInitialSync

serve.ts (separate entry OR same process in prod via dist/src/api/serve.js)
% % % Recursive setTimeout auto-sync loop (6h default)
% % % refreshExpiringMetaTokens
% % % pruneRawInsights + refreshCampaignHistoryRollups (24h interval)
% % % Coarse advisory lock: adlytic:auto-sync

Finding P1-01 — Monolithic route file | Medium
```

src/api/server.ts is ~3,392 lines containing auth, billing, Meta OAuth, sync, dashboard API, AI chat, admin, and 57 routes. This violates separation of concerns and increases merge conflict / review risk.

- Location: src/api/server.ts:1-3391
- Recommendation: Split into Hono sub-apps: routes/auth.ts, routes/meta.ts, routes/sync.ts, routes/billing.ts, mount in factory.

Finding P1-02 — Page/API boundary is clean but duplicated client logic | Low

Server pages emit HTML + inline IIFE scripts. Shared behavior lives in layout.ts SHARED_JS (apiFetch, pollSyncJob, sync status bar). dashboardPage.ts duplicates pollSyncJob for onboarding overlay (lines 1800-1811) instead of exclusively using window.pollSyncJob.

- Location: src/web/pages/dashboardPage.ts:1800-1811 vs src/web/layout.ts:729-741
- Risk: Divergent poll intervals, status handling, or max attempts over time.
- Recommendation: Onboarding should call window.pollSyncJob(jobId, { onProgress: updateOnboardingUI }).

Finding P1-03 — getAccount() assumes single ad account per workspace | High

```
const account = ws.adAccounts[0] ?? null;

• Location: src/api/server.ts:418-424
• Impact: Multi-account workspaces (schema allows many AdAccount rows) will silently operate on arbitrary first account (insertion order dependent).
• Recommendation: Explicit primary flag on AdAccount or deterministic orderBy: { createdAt: 'asc' } documented as canonical — already used in checkWorkspaceTokenHealth.ts:41-42 but not in getAccount().
```

Finding P1-04 — TypeScript any usage (31 hits) | Medium

Concentrated in:

File | Count | Examples

- src/services/getDashboard.ts | 12 | JSON/evidence shaping
- src/api/server.ts | 5 | safeJson Decimal, catch (e: any), Meta error body
- src/workers/runEngines.ts | 2 | Engine payloads

Engines (analytics, knowledge, rules, health, intelligence) | 7 | Evidence JSON

- Risk: Fragile runtime assumptions on Meta/Prisma/JSON shapes; refactors won't be caught by compiler.
- Recommendation: Introduce MetaErrorBody, PrismaJsonValue, zod schemas at API boundaries.

Finding P1-05 — Unhandled promise policy: log-only | Medium

serve.ts registers unhandledRejection logger but does not exit. Background setImmediate blocks use void (async () => { ... })() without top-level .catch() in some paths (relying on inner try/catch).

- Location: src/api/serve.ts:40-48, src/api/server.ts:3298-3316
- Risk: Silent partial failures if a non-awaited promise escapes inner try.
- Recommendation: Standardize runBackground(name, fn) wrapper with structured logging + metrics.

Finding P1-06 — process.env reads outside config.ts | Medium

Despite config.ts mandate, direct reads persist:

- src/api/server.ts:267 — ALLOWED_ORIGINS
- src/api/server.ts:1071-1074 — Stripe price, PUBLIC_APP_URL
- src/services/claudeClient.ts:21 — ANTHROPIC_API_KEY
- src/services/getDashboard.ts:347 — DASHBOARD_STAGE_TIMEOUT_MS
- src/services/stripeClient.ts, adminGuard.ts, mockMeta.ts, whatsappLink.ts
- Recommendation: Extend AppConfig with billing, ai, cors, admin sections; fail-fast in production.

Finding P1-07 — Standalone Prisma pool in getDashboard.ts | High

Module-level _pool + _standalonePrisma instantiated at import when DATABASE_URL is set.

- Location: src/services/getDashboard.ts:46-68
- Impact: CLI/scripts get a second connection pool (default max 10 connections each). Server correctly injects opts.prisma, but accidental import in hot path doubles pool pressure.
- Recommendation: Lazy-init standalone client; or require explicit createDashboardService(prisma).

Finding P1-08 — Positive: layer discipline | Strength

- metaClient.ts — transport only
- mappers/* — Meta !' normalized
- repositories/* — persistence contracts
- engines/* — downstream analytics (sync worker explicitly excludes these tables)
- getDashboard.ts — single product boundary DTO

This is better than typical MVP structure and should be preserved in refactors.

Pillar 2 — Concurrency, Locking & Data Integrity

2.1 Advisory lock implementation

Mechanism: FNV-1a-style 32-bit hash !' pg_try_advisory_lock(\$1) / pg_advisory_unlock(\$1).

- Location: src/lib/advisoryLock.ts:16-39
- Used by:

- Per-account sync: syncAccount.ts:189-317 (sync()), 1041-1254 (syncChunked()) - Auto-sync pass: serve.ts:224-234 (key adlytic:auto-sync)

Finding P2-01 — Session locks + pooled connections (fundamental) | Critical

Postgres advisory locks acquired via pg_try_advisory_lock(bigint) are session-scoped. Prisma \$queryRawUnsafe runs on a connection from pg.Pool and returns the connection to the pool after the query completes — but the lock remains on that backend session.

During a long syncChunked loop (minutes), subsequent queries may use different pooled connections, while the lock sits on the original connection until pg_advisory_unlock runs on that same connection.

The code comments acknowledge this (syncAccount.ts:1047-1060). Mitigation in syncChunked:

```
const [, lockRows] = await this.prisma.$transaction([
  this.prisma.$queryRawUnsafe(`SELECT pg_advisory_unlock_all()`),
  this.prisma.$queryRawUnsafe(`SELECT pg_try_advisory_lock($1)`, lockId),
]);
```

Problems:

1. sync() does NOT use unlock_all preflight — auto-sync in serve.ts:197 calls worker.sync(), not syncChunked. Crash mid-sync !' stale lock !' auto-sync silently skips account until connection recycled or manual intervention.

1. pg_advisory_unlock_all() is dangerous — releases every session advisory lock on that connection. If the same connection previously held adlytic:auto-sync coarse lock (unlikely but possible under pool reuse timing), it could release it early.

1. No pg_advisory_xact_lock anywhere in codebase (grep confirms zero hits). Transaction-scoped locks auto-release at COMMIT/ROLLBACK and are the recommended pattern for ORM pools — but they cannot span multi-minute chunk loops without holding a transaction open (bad for pool).

- Severity: Critical for ops reliability
- Recommendation (priority order):

1. Short term: Apply same unlock_all + acquire transaction to sync() OR migrate auto-sync to syncChunked with a minimal window. 2. Medium term: Dedicated pg.Client (non-pooled) for lock holder for duration of sync job. 3. Long term: Replace DB locks with SyncJob row-level locking (FOR UPDATE SKIP LOCKED) — schema already has SyncJob status machine.

Finding P2-02 — Advisory lock hash collision | Low

32-bit FNV hash of adAccountId cuid !' ~1/2³¹ collision probability between accounts. Unlikely at current scale but not zero.

- Location: src/lib/advisoryLock.ts:16-22
- Recommendation: Use pg_advisory_lock(hashtext(\$1)) on full string or two-int64 hashtextextended.

Finding P2-03 — SyncJob dedupe (Fix A) — good | Strength

Active job reuse prevents duplicate jobs and misleading "already in progress" UX.

- Location: src/api/server.ts:3234-3256
- Pairs with: f7b85af commit

Finding P2-04 — \$transaction batch size unbounded | High

DailyStatsRepo.upsertMany wraps one upsert per row in a single \$transaction:

- Location: src/repositories/dailyStatsRepo.ts:68-94
- Impact: 180-day backfill × multiple entities can create transactions with hundreds/thousands of statements !'

Postgres max_query_size, memory, or Prisma timeout; increased deadlock probability with concurrent engine reads.

• Recommendation: Chunk transactions (e.g. 50-100 upserts each); use createMany + ON CONFLICT raw SQL for bulk paths.

Finding P2-05 — Campaign reconcile transaction size | Medium

reconcileCampaignStatuses builds upserts array for all campaigns + optional updateMany in one \$transaction.

- Location: src/workers/syncAccount.ts:448-483
- Impact: Accounts with 500+ campaigns (Meta limit 200 per page but pagination exists in listCampaigns) could stress transaction.

Finding P2-06 — No deadlock retry | Medium

Prisma deadlocks (40P01) are not retried anywhere in sync or repos.

- Recommendation: Retry wrapper with exponential backoff on serialization/deadlock errors.

Finding P2-07 — Crash lock release claim is partially true | Medium

Comments state crash drops connection !' Postgres releases session locks. True only when the pool connection is actually torn down. Idle pooled connections may persist on PgBouncer transaction mode or Railway proxy — verify pool mode. pg_advisory_unlock_all fix addresses symptom, not root cause.

Finding P2-08 — Coarse auto-sync lock vs per-account lock interaction | Medium

serve.ts holds adlytic:auto-sync for entire pass (token refresh + all accounts serially). Duration may exceed SYNC_INTERVAL_MS if many accounts — mitigated by recursive setTimeout (not setInterval).

- Location: src/api/serve.ts:220-243
- Risk: Second instance skips entire pass (good); first instance holds lock for hours if account count grows.

Finding P2-09 — Positive: idempotent upserts | Strength

DailyStat unique on (entityType, entityId, date) — re-sync converges. Raw insights append-only by design (audit trail) with 90-day prune.

Pillar 3 — API Resilience & Meta Integrity

3.1 MetaClient transport layer

Finding P3-01 — Retry policy | Medium (acceptable with gaps)

- Retries: HTTP 429, 5xx, network errors — exponential backoff + jitter, max 5 retries
- Location: src/services/metaClient.ts:212-238
- Does NOT retry: Meta OAuthException code 17 (handled separately in sync worker)
- Does NOT retry: 4xx except 429 (correct — client errors)

Finding P3-02 — Code 17 handling | Strength

withCode17Retry sleeps 2s, retries once.

- Location: src/workers/syncAccount.ts:148-158
- Documented rationale: code 17 is per-ad-account, not IP-routable.

Finding P3-03 — Pagination cap | Medium

maxPages = 500 with warning log if truncated.

- Location: src/services/metaClient.ts:195-208
- Impact: Large accounts may silently truncate insights history in edge cases.

Finding P3-04 — Token lifecycle | High (managed)

Path | Behavior

refreshExpiringMetaTokens | Re-exchange USER_OAUTH tokens within 7d of expiry

Expired | PAUSE + null token

Decrypt fail | PAUSE (keep ciphertext)

Meta 190 | handleMeta190 — PAUSE legacy / NEEDS_REGRANT system user

System user | Skipped in refresh (non-expiring)

- Location: src/workers/refreshMetaTokens.ts, src/services/accountToken.ts:86-111
- Gap: tokenExpiresAt query in refresh has no index (see Pillar 5).

Finding P3-05 — Meta webhook signature verification | Strength

Check | Implementation

Raw body | c.req.raw.text() before JSON parse

HMAC-SHA256 | verifyMetaSignature with META_APP_SECRET

Timing-safe compare | Length guard + crypto.timingSafeEqual

Async processing | setImmediate + 200 OK

Debounce | 5s per account, timer.unref()

- Location: src/services/metaWebhook.ts:53-65, src/api/server.ts:1196-1222
- Severity: No Critical issues found in webhook auth path.

Finding P3-06 — Meta webhook idempotency | Medium (accepted design)

No per-delivery id — debounce coalescing only. At-least-once delivery may cause redundant reconciles.

- Location: src/services/metaWebhook.ts:20-23
- Mitigation: Reconcile is idempotent upsert; periodic auto-sync is safety net.

Finding P3-07 — Meta webhook GET handshake | Low

Simple string compare of verify token — adequate for subscription setup.

- Location: src/api/server.ts:1169-1183

- Note: hub.challenge returned as plain text — correct per Meta spec.

Finding P3-08 — Stripe webhook | Strength

Raw body, signature verify, transactional idempotency, 500 on handler crash (retry), 200 on business no-op.

- Location: src/api/server.ts:1114-1161, src/services/subscriptionService.ts

Finding P3-09 — Error parsing inconsistency | Medium

190 detection uses regex in some paths, typed MetaApiError.body.error.code in others:

- serve.ts:203 — regex on error string
- server.ts:3312 — body?.error?.code === 190
- Recommendation: Centralize isMetaErrorCode(err, 190) in metaClient.ts.

Finding P3-10 — Invalid breakdown combination handling | Strength

Non-fatal skip for OAuthException #100 on breakdown sync — prevents whole-job failure.

- Location: src/workers/syncAccount.ts:123-136

Finding P3-11 — OAuth session in-memory | High

Post-OAuth account picker session stored in oauthSessions Map — lost on redeploy / wrong instance.

- Location: src/api/server.ts:186-195, 2661+
- Contrast: CSRF state correctly in OAuthState table.
- Recommendation: Persist session in Redis or DB with TTL; or return accounts inline in encrypted callback cookie.

Finding P3-12 — META_APP_SECRET required for webhook but optional in config | Medium

Webhook returns 403 if absent — correct fail-closed. No boot-time warn beyond missing secret at request time.

Pillar 4 — UI/UX Robustness

4.1 Sync polling (commit 885f009 / 26858ed)

Finding P4-01 — Centralized pollSyncJob in SHARED_JS | Strength

- Polls GET /api/sync-jobs/:jobId every 1.5s, max 180 attempts (~4.5 min)
- Progress bar uses chunksDone/chunksTotal with fallback to progress field
- Location: src/web/layout.ts:721-741, 813-831

Finding P4-02 — runWorkspaceSync + resumeActiveSyncIfAny | Strength

- sessionStorage remembers active job id
- Handles reused: true from Fix A dedupe
- Disables sync buttons during poll
- Location: src/web/layout.ts:849-912

Finding P4-03 — Dashboard onboarding duplicate poll | Low

Local pollSyncJob in dashboardPage mirrors layout logic.

- Location: src/web/pages/dashboardPage.ts:1800-1811

Finding P4-04 — Status enum mismatch in resume check | Medium

resumeActiveSyncIfAny checks for RUNNING / IN_PROGRESS but schema enum is PENDING | PROCESSING | COMPLETED | FAILED.

- Location: src/web/layout.ts:892
- Impact: Harmless (dead branches) but signals copy-paste from another system; could mask future enum additions.

Finding P4-05 — Null-safe rendering | Strength (post-885f009)

- applyDashboardData: Array.isArray guards, safeRender per section
- Location: src/web/pages/dashboardPage.ts:1852-1905
- buildKpisFromInsights fallback when dashData.kpis empty
- Loading safety timeout 5s — startLoadingSafetyTimeout

Finding P4-06 — Token decrypt banner | Strength

- Probes `/api/workspaces/:id/token-health` on shell init
- Shows reconnect CTA; dismiss per workspace in `sessionStorage`
- Location: `src/web/layout.ts:1022-1088`, `src/services/checkWorkspaceTokenHealth.ts`

Finding P4-07 — Token health checks primary account only | Medium (intentional)

Only oldest account by `createdAt` asc — matches comment about orphan rows.

- Location: `src/services/checkWorkspaceTokenHealth.ts:38-52`
- Gap: If primary account has no token but secondary does, banner won't show.

Finding P4-07b — `checkWorkspaceTokenHealth` uncommitted local changes | Info

Git status shows modified `checkWorkspaceTokenHealth.ts` — audit reflects primary-account probe aligned with `getAccount` intent.

Finding P4-08 — CDN / Meta thumbnail fallback | Strength

- Real `<img>` with `onerror="creativeImgFailed(this)"`, emoji fallback, shimmer placeholder
- `referrerpolicy="no-referrer"` for CDN URLs
- Location: `src/web/pages/campaignsPage.ts:744-752`, `src/web/layout.ts:765-779`

Finding P4-09 — Meta payload edge cases in UI | Medium

- Breakdown Arabic maps with raw fallback for unknown Meta enum values — good
- Location: `src/web/pages/campaignsPage.ts:791+`
- Creative copy: `headline || primaryText || description` — empty shows em-dash
- Gap: Missing `object_story_spec` video thumbnail extraction when `thumbnail_url` absent — dashboard may show emoji for video ads frequently.

Finding P4-10 — `friendlyApiError` mapping | Strength

Maps decrypt, timeout, non-JSON, in-progress, permission errors to user Arabic/English messages.

- Location: `src/web/layout.ts:743-763`

Finding P4-11 — Dashboard refresh interval | Low

`setInterval(refreshDashboard, REFRESH_MS)` paused on `visibilitychange` — good battery/network citizenship.

- Location: `src/web/pages/dashboardPage.ts:1687-1700`

Finding P4-12 — Chart.js CDN dependency | Medium

CSP allows `cdn.jsdelivr.net` for scripts. Offline CDN !' charts broken, core KPIs still render.

- Location: `src/api/server.ts:299-300`

Pillar 5 — Security & Performance

5.1 Security

Finding P5-01 — JWT + `tokenVersion` revocation | Strength

DB check on every authenticated request via `getUserId`.

- Location: `src/api/server.ts:364-373`, `src/services/jwtAuth.ts`

Finding P5-02 — CORS defaults permissive in dev | High (if misconfigured prod)

When `ALLOWED_ORIGINS` empty !' origin: `'*` with `credentials: true` — browsers may restrict, but misconfiguration risk.

- Location: `src/api/server.ts:267-279`
- Recommendation: Production must set `ALLOWED_ORIGINS`; add `config.ts` fail-fast.

Finding P5-03 — CSP allows unsafe-inline scripts | Medium (inherent to architecture)

All page JS is inline. XSS in any unescaped user content is critical.

- Location: `src/api/server.ts:297-308`
- Mitigation: Pages use `escHtml` helpers — spot-check needed on AI chat rendering.

Finding P5-04 — TLS verification disabled for external DB | High

ssl: { rejectUnauthorized: false } for non-.railway.internal hosts.

- Location: src/api/serve.ts:67-74, getDashboard.ts:66
- Impact: MITM on DATABASE_URL connection in hostile network.
- Recommendation: Railway/Supabase CA pinning or sslmode=verify-full with proper CA.

Finding P5-05 — Rate limiting in-memory only | Medium

Login 10/15min, register 5/hour per IP — not shared across instances.

- Location: src/api/server.ts:144-163

Finding P5-06 — Platform admin via env email list | Medium

PLATFORM_ADMIN_EMAILS — acceptable for Phase 1; no RBAC audit trail beyond PaymentEvent.

Finding P5-07 — Token encryption | Strength

AES-256-GCM, random IV, auth tag, fingerprint logging, distinct error type.

- Location: src/services/tokenEncryption.ts

Finding P5-08 — AI context sanitization | Medium

dataSanitizer.ts exists (test file present) — LLM paths should be verified to always sanitize; out of scope for line-by-line proof but required for prod.

5.2 Performance & indexes

Finding P5-09 — Missing index: AdAccount.lastSyncedAt | High

Auto-sync queries filter status: ACTIVE + token conditions but no index on lastSyncedAt for stale-account prioritization or monitoring queries.

- Schema: prisma/schema.prisma:94 — only @@index([workspaceId])
- Recommendation: @@index([status, lastSyncedAt]) or partial index on ACTIVE.

Finding P5-10 — Missing index: AdAccount.tokenExpiresAt | High

Token refresh worker queries:

```
tokenExpiresAt: { not: null, lte: now }
```

No index !' sequential scan as account table grows.

- Location: src/workers/refreshMetaTokens.ts:74-97
- Recommendation: @@index([tokenSource, status, tokenExpiresAt])

Finding P5-11 — Missing index: Meta external IDs for webhook lookup | Medium

findAdAccount uses externalAccountId: { in: candidates } — unique constraint (platform, externalAccountId) exists !' OK.

Finding P5-12 — SyncJob polling index | Strength

@@index([adAccountId, status]) supports Fix A lookup.

- Schema: prisma/schema.prisma:810

Finding P5-13 — Raw insights growth | Medium

Append-only until daily prune (90d default). High-frequency sync × many accounts !' table bloat between prunes.

- Location: src/api/serve.ts:249-261
- Recommendation: Partial index on fetchedAt for prune query; consider partitioning.

Finding P5-14 — Connection pool defaults | Medium

pg.Pool() default max: 10 — not explicitly configured.

- Location: src/api/serve.ts:68-75
- Impact: 10 concurrent long syncs + HTTP traffic !' pool exhaustion / queue latency.
- Recommendation: Set max, idleTimeoutMillis, monitor pg.pool.totalCount.

Finding P5-15 — getDashboard stage timeout | Strength

9s default per stage via DASHBOARD_STAGE_TIMEOUT_MS — prevents hung dashboard.

- Location: src/services/getDashboard.ts:347+

Finding P5-16 — Memory: interval timers | Low

- setInterval for daily maintenance never cleared on shutdown (process exit OK)
- Meta webhook pendingReconciles Map — timers unref()'d — good
- OAuth prune only on access — no periodic prune timer for oauthSessions except on OAuth routes

Finding P5-17 — Platform stats cache | Low

In-memory cache in getPlatformStats.ts — per-instance inconsistency.

Cross-Cutting Observations

Environment variables inventory (production-critical)

Variable | Validated in config.ts | Required prod

DATABASE_URL | Yes (fail) | Yes
JWT_SECRET | Yes (fail, min 32) | Yes
TOKEN_ENCRYPTION_KEY | Yes (fail, 64 hex) | Yes
META_APP_ID/SECRET | Optional | Yes for OAuth/webhooks
META_VERIFY_TOKEN | Optional | Yes for webhook subscribe
ALLOWED_ORIGINS | No | Yes
STRIPE_* | Partial (runtime) | If billing enabled
ANTHROPIC_API_KEY | No | If AI enabled
PUBLIC_APP_URL | No | Yes for Stripe redirects

Test coverage vs production paths

Test scripts (test_worker.ts, test_health.ts, etc.) cover logic modules but no automated integration test for:

- Advisory lock + pool interaction
- Webhook HMAC verification
- SyncJob full lifecycle
- OAuth state DB roundtrip

Workers not extracted

runNarrationWorker.ts, brainNarrationCron.ts, rollupHistory.ts exist as separate entry points but core sync runs in API process — scaling concern.

Prioritized Recommendations

P0 — Before multi-instance / paid SLA

| Action | Pillar | Effort

R1 | Fix advisory lock strategy: dedicated non-pooled client OR migrate auto-sync to syncChunked + apply unlock preflight to sync() | P2 | M
R2 | Persist OAuth post-callback session (replace oauthSessions Map) | P3 | M
R3 | Add indexes: (tokenSource, status, tokenExpiresAt), (status, lastSyncedAt) on ad_accounts | P5 | S
R4 | Configure pg.Pool max + timeouts; document total DB connection budget | P5 | S
R5 | Set ALLOWED_ORIGINS + config.ts fail-fast in production | P5 | S

P1 — Next quarter

| Action | Pillar | Effort

R6 | Extract sync/engine work to job queue (BullMQ, pg-boss, or Railway worker service) | P1/P2 | L
R7 | Chunk \$transaction in DailyStatsRepo.upsertMany | P2 | S
R8 | Split server.ts into route modules | P1 | M
R9 | Centralize Meta error code helpers | P3 | S
R10 | Align getAccount() with primary account selection + document multi-account roadmap | P1 | S

P2 — Quality / debt

| Action | Pillar | Effort

R11 | Reduce any usage with typed Meta/JSON interfaces | P1 | M

R12 | Deduplicate dashboard pollSyncJob !' use SHARED_JS | P4 | S
R13 | TLS verify-full for DATABASE_URL | P5 | S
R14 | Integration tests for webhooks + SyncJob lifecycle | All | M
R15 | Vendor Chart.js or self-host CDN script | P4/P5 | S
Effort key: S = small (hours), M = medium (days), L = large (week+)

Master-Level Verdict — Biggest Risks

Top 3 Critical / High Headlines

- 1. Session advisory locks on pooled Prisma connections — Can cause false "sync already in progress," skipped auto-syncs, and stuck SyncJobs after deploy/OOM; syncChunked mitigation incomplete for sync() auto-sync path.
- 1. Single-process background workload coupling — Sync, engines, brain, webhooks, and HTTP share one event loop and DB pool; no queue isolation under load.
- 1. Multi-instance statefulness — In-memory OAuth sessions, rate limits, and caches break horizontal scaling; CSRF state fix proves team knows the pattern but hasn't finished applying it everywhere.

Architectural maturity scorecard

Area | Grade | Notes
Domain layering | A- | Clear Meta/DTO/engine separation
Data integrity (ETL) | B+ | Idempotent upserts; transaction sizes risk
Concurrency | C | Advisory locks + pool = fragile
API security (webhooks) | A- | Meta + Stripe correctly verified
UI resilience | B+ | Strong post-885f009; minor duplication
Ops readiness | C+ | Config centralization good; indexes/logging gaps
Scale readiness | D+ | Single-instance assumptions predominate

Final statement

Adlytic demonstrates above-average engineering discipline for an ads analytics MVP — particularly Meta integration boundaries, sync UX honesty (real polling vs fake "sync started" toasts), and webhook security parity with Stripe. The codebase is not reckless; recent commits show responsive hardening.

The gap between "works on Railway single dyno" and "production-grade multi-tenant SaaS" is concentrated in job orchestration and Postgres locking semantics, not in UI polish or Meta API parsing. Address R1–R5 before marketing uptime SLAs or running "e2 replicas."

Appendix A — Key file reference map

File | Role
src/api/serve.ts | Process entry, pool, auto-sync loop, maintenance
src/api/server.ts | Hono routes (57), OAuth, sync API, webhooks
src/workers/syncAccount.ts | ETL orchestrator, advisory locks, SyncJob driver
src/services/metaClient.ts | Meta Graph transport + retry
src/services/metaWebhook.ts | HMAC verify + debounced reconcile
src/lib/advisoryLock.ts | FNV hash + try/unlock helpers
src/web/layout.ts | SHARED_JS: apiFetch, pollSyncJob, sync UI
src/web/pages/dashboardPage.ts | Dashboard SSR + onboarding poll
src/services/checkWorkspaceTokenHealth.ts | Token decrypt probe
src/config.ts | Env validation + boot checklist
prisma/schema.prisma | 30+ models, SyncJob, MetaConnection

Appendix B — SyncJob state machine

PENDING !' PROCESSING !' COMPLETED
!~ FAILED

Progress fields updated during syncChunked: chunksTotal, chunksDone, progress, cursorDate, rowsFetched, rowsUpserted.

Appendix C — Audit tooling

- PDF generation: `npx tsx scripts/compileMasterAuditPdf.ts`
- Output: `Adlytic_Master_Software_Audit.pdf`

End of ADLYTIC_MASTER_ARCHITECT_AUDIT_2026

